

MXCuBE CUSTOM TASKS

Quite confusing, quite technical

Paweł Moćko

2026-08-20

Custom tasks allow us to write custom experiments in MXCuBE that may be specific to our site, or would be tricky to share with others. They allow us to:

- avoid modifying shared code
- develop our own experiments without appeasing other facilities ;)
- they can be easily tweaked by the beamline staff if need be

To create a new form, there are two important pieces to implement:

- **DATA_MODEL** — a Pydantic model describing:
 - data that is supposed to come from the web app
 - what form fields will be displayed in the web app & their constraints
 - form layout
- **QueueEntry** — a derived class from `BaseQueueEntry`, needs 3 methods:
 - `pre_execute` — setup for the task
 - `execute` — actual logic to perform the task
 - `post_execute` — cleanup after the task is finished / fails.

Creating your own task

SOME BOILERPLATE – DATA MODEL

Just a plain Pydantic model :)

```
from pydantic import BaseModel, Field

class NewCollectionTaskParameters(BaseModel):
    num_images: int = Field(...) # ellipsis marks required fields.
    exp_time: float = Field(...)
```

Just a plain Pydantic model :)

```
from pydantic import BaseModel, Field

class NewCollectionTaskParameters(BaseModel):
    num_images: int = Field(...) # ellipsis marks required fields.
    exp_time: float = Field(...)
    # Fun Fact: currently MXCuBE-Web always assumes that these 2 fields exist
    here :D
```

SOME BOILERPLATE – DATA MODEL

Which we can annotate:

```
class NewCollectionTaskParameters(BaseModel):
    num_images: int = Field(
        ..., ge=0, description="number of images taken during data collection"
    )
    exp_time: float = Field(
        default=100e-6,
        gt=0,
        lt=1,
        unit="s",
        description=(
            "Amount of time the crystal is exposed to the beam when"
            "collecting a particular image."
        ),
    )
```

And we need to put some properties that are always required...

```
class NewCollectionDataModel(BaseModel):  
    path_parameters: PathParameters  
    common_parameters: CommonCollectionParamters  
    collection_parameters: StandardCollectionParameters  
    user_collection_parameters: NewCollectionTaskParameters  
    legacy_parameters: LegacyParameters
```


And we need to put some properties that are always required...

```
class NewCollectionDataModel(BaseModel):
    path_parameters: PathParameters
    common_parameters: CommonCollectionParameters
    collection_parameters: StandardCollectionParameters
    user_collection_parameters: NewCollectionTaskParameters
    legacy_parameters: LegacyParameters
    # and there is one required bit, to be explained later :)
    @staticmethod
    def update_dependent_fields(field_data: NewCollectionTaskParameters):
        return {}
```

This is quite a weird detail.

For each `QueueEntry`, there needs to be a `QueueModel` class in a 1:1 correspondence, usually it looks like this:

```
from mxcube.core.model.queue_model_objects import DataCollection

class NewCollectionQueueModel(DataCollection):
    pass
```

SOME BOILERPLATE – WIRING IT ALL TOGETHER

```
# new_collection.py  
from mxcube.core.queue_entry.base_queue_entry import BaseQueueEntry  
  
# class name has to match filename  
class NewCollectionQueueEntry(BaseQueueEntry):  
    NAME = "New Collection" # name in the context menu
```

SOME BOILERPLATE – WIRING IT ALL TOGETHER

```
from mxcubequeue_entry.base_queue_entry import BaseQueueEntry,  
TaskPrerequisite
```

```
class NewCollectionQueueEntry(BaseQueueEntry):  
    NAME = "New Collection"  
    REQUIRES = [  
        TaskPrerequisite.POINT,
```

SOME BOILERPLATE – WIRING IT ALL TOGETHER

```
from mxcubequeue_entry.base_queue_entry import BaseQueueEntry,  
TaskPrerequisite
```

```
class NewCollectionQueueEntry(BaseQueueEntry):  
    NAME = "New Collection"  
    REQUIRES = [  
        TaskPrerequisite.POINT,  
        # Trivia time! What does it mean?!  
        TaskPrerequisite.NO_SHAPE_2D,  
    ]
```

SOME BOILERPLATE – WIRING IT ALL TOGETHER

```
from mxcube.core.queue_entry.base_queue_entry import BaseQueueEntry,  
TaskPrerequisite
```

```
class NewCollectionQueueEntry(BaseQueueEntry):  
    NAME = "New Collection"  
    REQUIRES = [  
        TaskPrerequisite.POINT,  
        # Trivia time! What does it mean?!  
        TaskPrerequisite.NO_SHAPE_2D,  
    ]  
    QMO = NewCollectionQueueModel
```

SOME BOILERPLATE – WIRING IT ALL TOGETHER

```
from mxcube.core.queue_entry.base_queue_entry import BaseQueueEntry,  
TaskPrerequisite
```

```
class NewCollectionQueueEntry(BaseQueueEntry):  
    NAME = "New Collection"  
    REQUIRES = [  
        TaskPrerequisite.POINT,  
        # Trivia time! What does it mean?!  
        TaskPrerequisite.NO_SHAPE_2D,  
    ]  
    QMO = NewCollectionQueueModel  
    DATA_MODEL = NewCollectionDataModel
```

SOME BOILERPLATE – WIRING IT ALL TOGETHER

Then enable it in the `beamline.yaml` config file:

```
# ...
```

```
available_methods:  
  new_collection: true  
# ...
```

the above key matches the name of the file containing `NewCollectionQueueEntry`

SOME BOILERPLATE – WIRING IT ALL TOGETHER

And we should get something like this:

New Collection

Path:

/tmp/visitor/localuser/mxcubewebtest/20260814/RAW_DATA/d/d-dsa/[RUN#]

Filename:

d-dsa_[RUN#]_[IMG#]

Sample name

d/d-dsa/

Prefix

d-dsa

Acquisition

Exposure Time [s]

0.0001

Num Images

0

Default Parameters

Run Now

Add to Queue

DEFINING UI LAYOUT

To change looks of a form, we can write a `ui_schema` method. the customization options include:

- order of inputs
- widgets used for inputs (e.g. dropdown menu, text input, number input, ...)
- layout (width of a particular input)
- grouping inputs
- splitting groups of inputs into rows (MAX IV only code, it will definitely change “soon”)

```
class NewCollectionDataModel(BaseModel):  
    # ...  
    @staticmethod  
    def ui_schema():  
        return json.dumps({  
            "ui:order": [ "num_images", "exposure_time"],  
        })
```

DEFINING UI LAYOUT – EXAMPLE 2

```
@staticmethod
def ui_schema():
    processing_group = {"group": "Processing"}
    col_4 = {"col": 4}
    processing_ui_options = {"ui:options": {**processing_group, **col_4}}
    return json.dumps(
        {
            "cell_a": processing_ui_options,
            "cell_b": processing_ui_options,
            "cell_c": processing_ui_options,
            "cell_alpha": processing_ui_options,
            "cell_beta": processing_ui_options,
            "cell_gamma": processing_ui_options,
        }
    )
```

We may use the aforementioned `update_dependent_fields` function to modify values of a particular field based on value of the other fields. So we may define a field in the parameters model:

```
# ...
illumination_time: float = Field(
    default=round(DEFAULT_NUM_IMAGES * DEFAULT_EXPOSURE_TIME, 6),
    unit="s",
    description="Total amount of time the point is illuminated.",
)
```

DERIVED PARAMETERS

And update it whenever any of the form fields changes:

```
# ...  
@staticmethod  
def update_dependent_fields(  
    field_data: dict[str, Any], updated_field_name: str | None  
) -> dict[str, Any]:  
    illumination_time = field_data.get("num_images", 0) * field_data.get(  
        "exposure_time", 0.0  
    )  
  
    return {"illumination_time": round(illumination_time, 6)}
```

The second parameter to `update_dependent_fields` is...

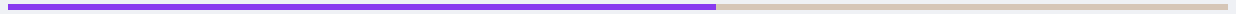
MAX IV ONLY

It is handy for some bidirectional dependencies, eg. $A \propto C \times B$ which was the case for HVE task form.

We may also make it readonly in the UI

```
@staticmethod  
def ui_schema():  
    return json.dumps({"illumination_time": {"ui:disabled": True}})
```

The cookhouse



to have some quick example for what we could use `pre_execute` and `post_execute` we may imagine that the task would leave some output that we can later retrieve from somewhere else.

```
def connect_to_the_cookhouse():  
    log.info("Cookhouse: knocking on the door at %s", COOKHOUSE_ADDRESS)  
    return {"address": COOKHOUSE_ADDRESS, "connected": True}  
  
def fetch_results_from_the_cookhouse():  
    log.info("Cookhouse: fetching the results")  
    return {"tea_of_the_day": "Dominika's new shirt", "number_of_cups": 1}  
  
def leave_the_cookhouse():  
    log.info("Cookhouse: leaving, door closed")
```

ADDING SOME BEHAVIOUR

For now let's establish a connection to some external server we would need before the task

```
def pre_execute(self):  
    super().pre_execute() # sets a `running` flag essentially  
    connect_to_the_cookhouse()
```

Having a `pre_execute` allows us to have a clean place to put some “prerequisites” like some kind of persistent connection or other kind of resource...

There is also a more technical reason related to how is a tree of “TaskNodes” managed by the queue, but that's not for today.

There is also a sibling `post_execute` method, using it may lead to some confusing cleanup logic. You may cope without it if you follow soft guideline.

First, some convention. To ensure cleanup will occur, such struture is reccomended.

```
def execute(self):  
    super().execute()  
    try:  
        self._do_the_task()  
    finally:  
        self._cleanup()
```

THE TASK PROPER

And for a simple implementation: let's move to a point, open shutters, sleep for a bit and enjoy a cooked sample.

```
def _do_the_task(self):  
    data = self.get_data_model()  
    shape_id = data.task_data.collection_parameters.shape
```

THE TASK PROPER

And for a simple implementation: let's move to a point, open shutters, sleep for a bit and enjoy a cooked sample.

```
def _do_the_task(self):  
    data = self.get_data_model()  
    shape_id = data.task_data.collection_parameters.shape  
    point = HWR.beamline.sample_view.get_shape(shape_id)  
    point_name = point.id  
    HWR.beamline.diffractometer.move_to_centred_position(  
        point.get_centred_position()  
    )  
  
    duration = data.task_data.user_collection_parameters.illumination_time  
    HWR.beamline.collect.open_safety_shutter()  
    HWR.beamline.collect.open_fast_shutter()  
    gevent.sleep(duration)
```

HOW LONG WILL IT TAKE?

```
@staticmethod  
def get_estimated_time(field_data: dict[str, Any]) -> int:  
    illumination_time = field_data.get("illumination_time", 0)  
  
    return math.ceil(illumination_time + 1)
```

MAX IV ONLY

HOW LONG WILL IT TAKE?

New Collection

Path:

/data/visitors/biomax/20170251/20260819/raw/dsasd/dsasd-a/[RUN#]

Filename:

dsasd-a_[RUN#]_[IMG#]

Sample name

dsasd/dsasd-a/

Prefix

dsasd-a

Acquisition

Exp Time [s]

0.002

Illumination Time [s]

6

Num Images

12

Default Parameters

Estimated Time: 00:00:07

Run Now

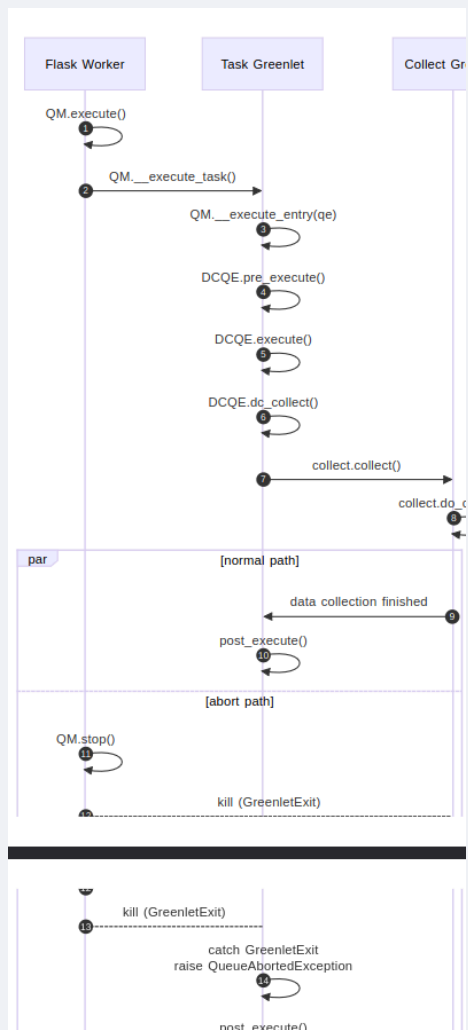
Add to Queue

In MXCuBE, a task may not be able to end execution because:

- it has been aborted
- the queue has been stopped before execution attempt, so it has been skipped.
- failure during execution

In theory `BaseQueueEntry` defines methods like `post_excute` and `stop()` which should handle post-task cleanup and cases of task being aborted. Sadly sometimes it is hard what will predict if one were to go with this route...

CLEANUP



Just sticking with `execute` for that kind of task is perfectly reasonable.

A QUICK RECIPE

1. Define the task parameters (pydantic model)
2. Do the boilerplate around new queue entry
3. Define what will `execute` do
4. ... and wrap it in a `try/finally` block to ensure some cleanup

- On the upstream version of mxcube, the custom tasks are currently broken.
- There is currently no way of showing custom errors / warnings. Only generic ones like “num_images has to be > 0”.
- Creating a wrapper around a “standard” Data Collection is **tricky**... (and should not be done.)
- Implicit conventions from non-generic tasks that have to be hold (i.e. naming of some fields..)

SOME NEXT STEPS...

1. Fix after the rebase :D
2. Upstream maxiv-only stuff
3. Remove boilerplate / some weird assumptions
4. ...